

SLE-3: ARCHITECTURAL DESIGN USING FULL C4 MODEL

BFS / DFS Maze Search System

Course: 02AML204 – Introduction to Artificial Intelligence

Name: Shreya Bhosale | PRN: 25UAM140

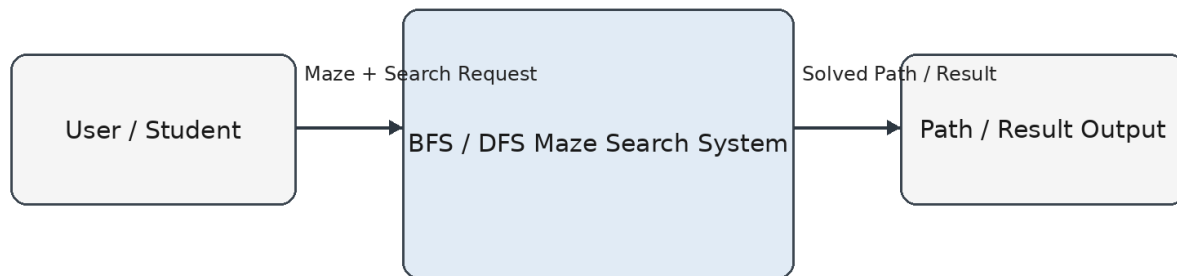
GitHub: <https://github.com/shreyabhosale2912-debug>

1. System Title & Short Description

The BFS/DFS Maze Search System is a search-based AI system continued from SLE-2. It accepts a maze, explores valid cells using Breadth-First Search (BFS) or Depth-First Search (DFS), and produces a path/result when the goal is reached. The system also supports performance comparison of the search approaches. The architecture below presents the same system using all four levels of the C4 model.

2. Context Diagram — Level 1

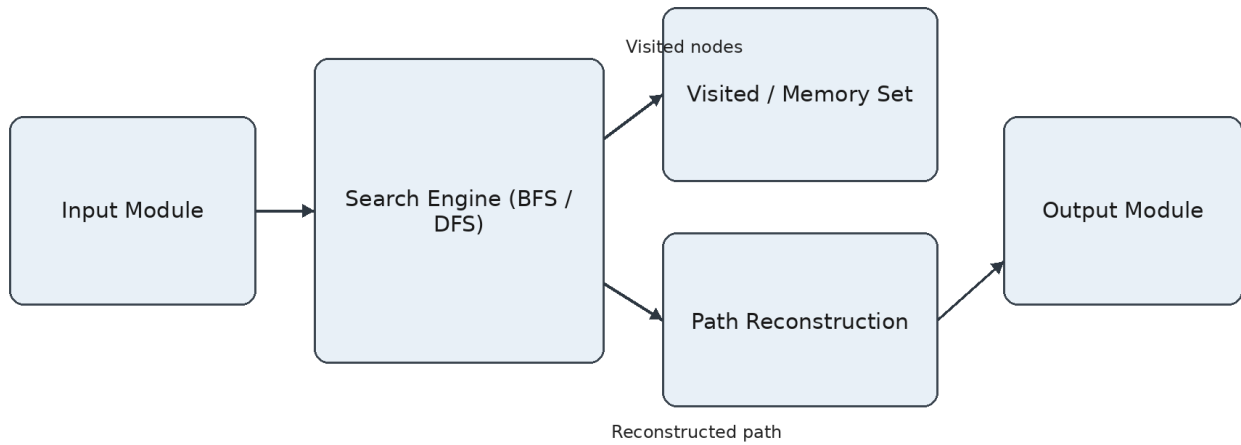
C4 Level 1 — Context Diagram



The user provides the maze and selects a search approach. The Maze Search System performs the search and returns the resulting path or search result. The context level focuses on the system boundary and its interaction with the user.

3. Container Diagram — Level 2

C4 Level 2 — Container Diagram

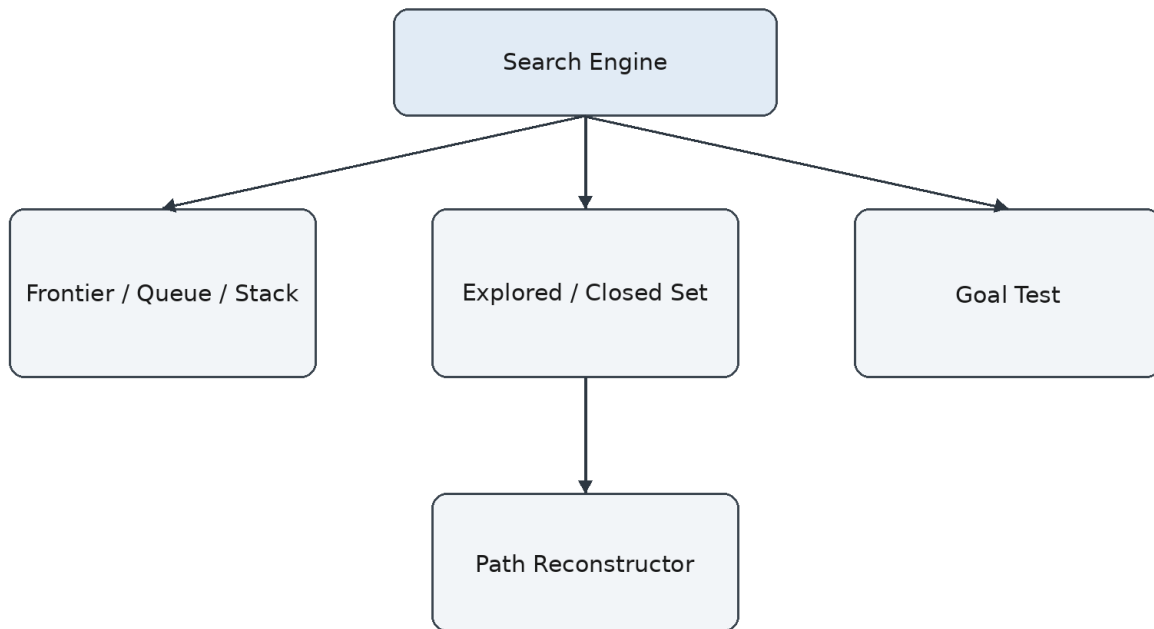


Container	Responsibility
Input Module	Takes the maze and search request as input.
Search Engine	Runs BFS or DFS to explore the maze.
Visited / Memory Set	Stores visited nodes/cells to avoid repeated exploration and loops.
Path Reconstruction	Uses search information to construct the final path.
Output Module	Displays the final path or search result.

4. Component Diagram — Level 3

The Search Engine is selected as the main container. Its internal components are shown below.

C4 Level 3 — Component Diagram: Search Engine



The Frontier component controls which cells are explored next: BFS uses a queue while DFS uses a stack/recursive traversal. The Explored/Closed Set tracks already visited cells. The Goal Test checks whether the target cell has been reached, and the Path Reconstructor builds the final route from the stored search information.

5. Code Level Overview — Level 4

C4 Level 4 — Code Level Overview

maze	Stores the 5x5 maze and wall/path information.
get_neighbors()	Returns valid neighboring cells.
bfs()	Searches the maze level-by-level using a queue.
dfs()	Searches the maze using depth-first traversal.
reconstruct_path()	Builds the final path from parent information.
profiler / timing	Measures search performance for comparison.

6. Design Decisions

The SLE-3 architecture continues the same maze/search system used in SLE-2, keeping the work consistent across the SLEs.

The system is separated into input, search, memory, path reconstruction and output responsibilities so that each part has a clear role.

The Search Engine is selected for the Component level because BFS/DFS is the core decision-making part of the system.

7. AI Contribution Note

AI tools used: ChatGPT / AI-assisted development tools.

What AI helped with: Understanding the C4 model, organizing the architecture, and improving the presentation of the report.

What I did myself: Selected the SLE-2 maze/search system, worked with the BFS/DFS implementation, checked the architecture, and prepared the final submission.

8. Conclusion

This SLE-3 helped me understand how a working AI search program can be represented as a complete software architecture. Using the C4 model, I represented the BFS/DFS Maze Search System from the overall context down to its main code-level functions. The architecture also makes the responsibilities and design choices of the system easier to understand and explain.